

函数

主要内容

- 函数定义：
 - 文档字符串
 - 参数传递
 - return语句
- 函数的参数：
 - 位置实参和关键字实参（格式化时已经介绍）
 - 位置形参和默认值形参
 - Position-only参数
 - 可变长参数
 - Keyword-only参数
- 名字空间：
 - 内置、全局和本地名字空间
 - 赋值类语句、global和nonlocal语句
 - 未限定名的名字空间搜索：LEGB
- 类型提示

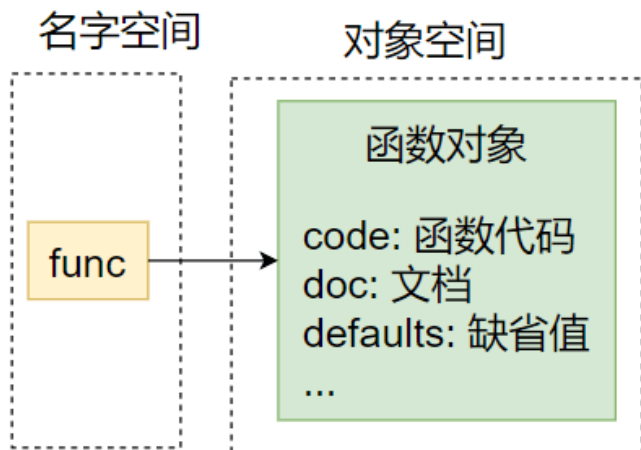
函数定义

def语句定义(define)函数

- 函数体在函数头部的下一行，一般整体缩进4个空格
- 如果函数体非常简单，特别是只有一条语句时，也允许与函数头部在同一行
- 创建函数对象
 - def语句描述的函数体代码、文档字符串、默认(缺省)值和闭包等信息保存在函数对象中
 - 函数体中的代码（包括参数传递）并没有执行
- 当前名字空间引入一个名字与新创建的函数对象绑定

```
def funcname(parameter_list):  
    '''docstring''' # optional  
    suite  
  
funcname(argument_list)
```

$$f(x) = \begin{cases} x^2 & x \geq 0 \\ -x & x < 0 \end{cases}$$



```
def f(x):  
    """上述数学函数f(x)的实现"""  
    if x >= 0:  
        return x ** 2  
    else:  
        return -x  
  
t = f(2)  
print(t)
```

functions.py

函数定义：一等对象

```
max_word = max('apple', 'python programming', 'java', key=len)
print(max_word) # 输出: python programming
```

函数也是一等对象(First-class Object)，和其他对象一样进行操作

- 可通过赋值语句给函数对象取另外一个别名：比如func=distance
- 可作为容器对象中的元素
- 可作为另外一个对象的属性
- 调用某个函数时可以 将函数对象作为参数 传递。比如max(..., key=len)
- 函数内部也可以定义函数（称为嵌套函数），可 返回另外一个函数对象
- 如果同一个函数名有多个定义时，在访问该名字时之前的最后一个函数定义有效

```
def quadratic_function(a, b, c):
    """一元二次函数: f(x) = ax^2 + bx + c """
    if a == 0:
        raise ValueError("a cannot be zero")

    def f(x):
        return a * x**2 + b * x + c

    return f

f = quadratic_function(2, 3, -5)
print(f(2)) # 输出: 9
```

```
def distance(x, y):
    """返回直角坐标系中点(x, y)到原点的距离
    传递的参数: x, y为点的X轴和Y轴坐标
    """
    x2 = x * x
    y2 = y * y
    return (x2 + y2) ** 0.5

func = distance
func(1, 1) # 等价于调用distance(1, 1)
```

```
def f1(): print("f1")

t = f1()
print(t)

def f1():
    return "f1"

t = f1()
print(t)
```

functions.py

函数定义：文档字符串

```
def funcname(parameter_list):  
    """docstring""" # optional  
    suite
```

- 文档字符串：提供友好的提示和使用帮助
 - 如果一个函数的**函数体**的第一个语句为字符串字面量，则该字符串会保存在**函数对象的__doc__属性**中
 - 如果一个**模块源程序**的第一个语句为字符串字面量，则该字符串会保存在**变量__doc__**中
 - 注意作为函数体时，字符串表达式的最开始必须要缩进，后续行则并没有强制，但建议缩进
- help()函数传递的是**函数对象**时会显示函数对象的文档字符串，而传递的是**模块对象**时显示模块对象的文档字符串以及相关函数的文档字符串
 - 每行前面的多余的空格类字符被移走

```
>>> import distance  
>>> help(distance)  
>>> help(distance.distance_to_origin)  
Help on function distance_to_origin in module distance:  
  
distance_to_origin(x, y)  
    返回直角坐标系中点(x, y)到原点的距离  
    传递的参数: x, y为点的X轴和Y轴坐标
```

```
"""计算某点到原点距离以及两点之间的距离"""
```

```
import math
```

```
def distance_to_origin(x, y):  
    """返回直角坐标系中点(x, y)到原点的距离  
    传递的参数: x, y为点的X轴和Y轴坐标  
    """  
    distance = math.sqrt(x**2 + y**2)  
    return distance
```

distance.py

函数的优点

- 在实际开发中，有许多操作是完全相同或非常相似的，仅仅是要处理的数据不同
 - 通过将可能需要反复执行的代码封装为函数，实现**代码的复用**
 - 当需要对这些反复执行的代码部分进行修改时，只需修改函数的实现即可
 - 问题分解：将问题分解为可管理的部分，提高了**可维护性**
 - 每个函数解决一个相对独立的问题，不要太复杂
 - 不需关心函数内部实现的具体细节，关注问题的高层次逻辑
 - 函数允许使用自描述的名称，提高了**可读性**
- 首先声明(定义)函数：
 - **描述**了解决某个问题需要执行的语句集合
 - 实现者独立进行设计、实现、测试和调试
- 调用函数：**执行**函数中包含的语句。使用者不需要关心具体实现的细节

问题分解示例

- 问题分解后可快速搭建整体框架，编写测试代码
- 函数的函数体中必须有一个语句
 - **pass语句**，空语句，表示什么也不做
 - 字符串字面量定义（文档字符串），描述函数的功能

functions.py

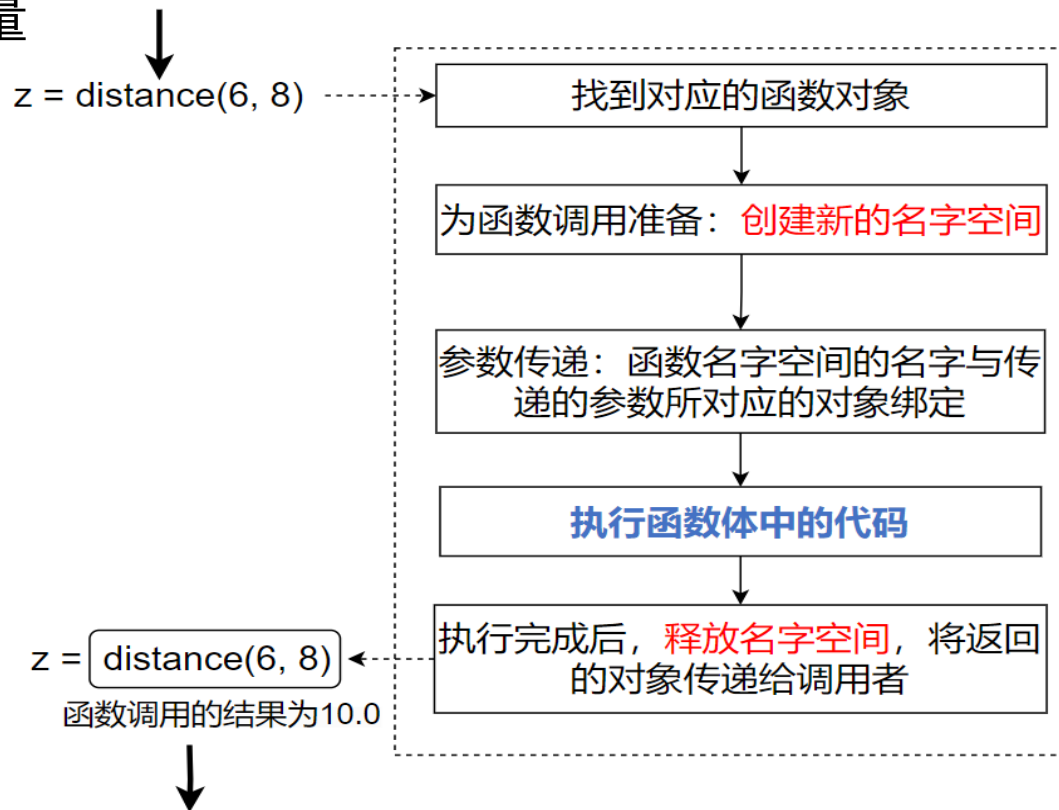
```
def prepare():  
    """从文件中读取相应的数据"""  
  
def calc(data):  
    pass  
  
def output_result(result):  
    pass  
  
if __name__ == "__main__":  
    data = prepare()  
    result = calc(data)  
    output_result(result)
```

函数调用：参数传递

参数传递：实参(Actual Argument) 与形参(Formal Parameter)对应，传递实参给出的对象到相应形参

- 采用**赋值传递(pass by assignment)**的策略：本地名字空间的形参变量=调用者传递的实参对象
- 为了避免side effect，函数调用时为本次调用**创建一个新的名字空间（本地名字空间）**
 - 形参变量为本地名字空间的**局部变量**，其使用范围为当前函数的函数体
 - 函数体内**赋值类语句**引入的名字一般也是局部变量

```
def distance(x, y):  
    """返回直角坐标系中点(x, y)到原点的距离  
    传递的参数：x, y为点的X轴和Y轴坐标  
    """  
    x2 = x * x  
    y2 = y * y  
    return (x2 + y2) ** 0.5  
  
z = distance(6, 8)
```



return语句

- 函数体内可包含return语句： `return [expression_list]`
 - 在调用函数执行函数体的代码时，如果执行的是return语句，结束函数的执行并返回
 - 关键字return后的表达式计算后的结果作为函数的返回值，即函数调用运算的结果
 - return后面如果没有表达式，则等价于 `return None`
- 函数体内可以没有return语句
 - 如果执行到函数体结束也无return语句，则等价于最后执行语句 `return None`
- return None：经常意味着函数的返回值没有什么特殊的意义
- 除非在函数调用的过程中出现了异常，函数一定会有返回值，只是有的函数可能返回值为None

```
def max1(x, y):  
    if x > y:  
        return x  
    else:  
        return y  
def max2(x, y):  
    if x > y:  
        print(x)  
    else:  
        print(y)
```

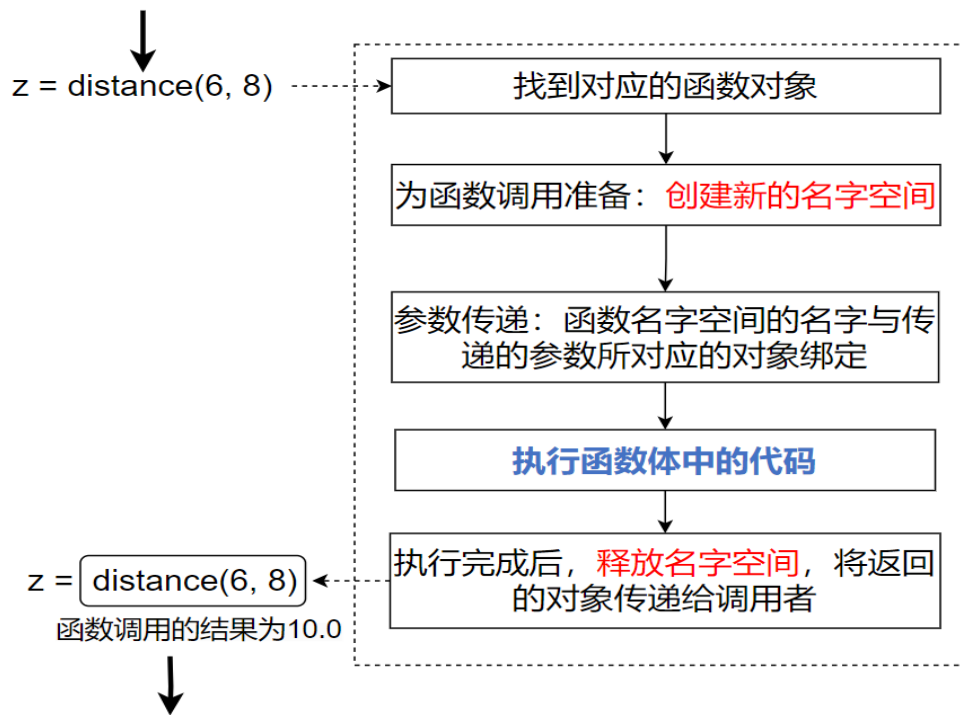
maximum.py

```
>>> max1(2, 4)  
4  
>>> max2(2, 4) # 返回值为None  
4  
>>> print(max1(2, 4))  
4  
>>> print(max2(2, 4)) # 返回值为None  
4  
None
```

交互式console：如果执行的语句为表达式obj，且其值**不为None**时调用 **`print(repr(obj))`**

函数的3个基本要素

- 传递的参数：赋值传递
- 返回的对象：return语句
- 要完成的功能：
 - 函数调用者把函数看成一个黑盒子，不用关注内部的实现细节，只要传递相应的参数，就应该完成相应的功能，最后返回一个对象
 - 函数的实现者关注的是如何实现这个黑盒子，每个参数是什么，怎么使用这些参数实现功能，怎么样通过return语句返回函数调用的结果
- 函数定义时，只是创建了一个函数对象
- 函数调用时，才真正执行函数体的代码
- 函数体内可以再调用其他函数，这些函数再调用其他函数
 - 在函数体内部甚至可以调用自己（称为**递归函数**）
 - 绝大部分情况下，各次函数调用之间是独立的。系统会保存函数调用时的状态（通过堆栈stack frame），在调用完成之后移除该次调用时的状态信息
 - python能够保证函数调用按照预定的顺序执行，而不会迷失



主要内容

- 函数定义：
 - 文档字符串
 - 参数传递
 - return语句
- **函数的参数：**
 - 位置实参和关键字实参（格式化时已经介绍）
 - 位置形参和默认值形参
 - Position-only参数
 - 可变长参数
 - Keyword-only参数
- 名字空间：
 - 内置、全局和本地名字空间
 - 赋值类语句、global和nonlocal语句
 - 未限定名的名字空间搜索：LEGB
- 类型提示

函数的参数

- 函数定义时，参数（形参）可以是：
 - 普通(位置)参数
 - 默认值参数，相应位置没有参数传递时使用默认值
 - Position-only参数(/之前的参数)：只能通过位置参数(实参)方式传递
 - 可变长度位置参数 *args，调用时相应位置可传递0个或者多个位置参数（实参）
 - keyword-only参数，出现在可变长位置参数之后的参数。如果要传递值时只能通过关键字参数（实参）方式传递
 - 可变长度关键字参数 **kwargs，调用时可传递0个或多个关键字参数（实参）
- 函数调用时，参数（实参）可以是：
 - 普通（位置）参数
 - 关键字参数

```
def func(a, b, /, c, x='x', y='y'):  
    print(a, b, c, x, y)  
  
func(1, 2, 3, y=5)
```

函数定义：位置形参和默认值形参

- **位置参数**：仅包含形参名的参数
- **缺省值或默认值参数**

```
def func(a, b, c, x='x', y='y'):  
    print(a, b, c, x, y)
```

```
x = 5  
def f1(a, x=x, y=x+5):  
    print(a, x, y)  
print(f1.__defaults__)  
# 输出 (5, 10)
```

- arg=default 给出了形参的名字以及默认值
- “**特殊的位置参数**”，调用时可以不传递值，此时形参对应应在函数定义时指定的默认值对象
- **函数对象的属性__defaults__**：元组类型，保存了所有默认值参数的当前值
- arg为函数调用时创建的**新名字空间内部的名字**，表达式default是在**函数定义时所在名字空间**处进行计算的
- 如果默认值参数的默认值为不可变对象，默认值无法更改，不会有什么问题
- 如果默认值参数的默认值为**可变对象，则需要特别注意（以后会展开）**
- 默认值参数右边不能再有位置参数：即最前面是位置参数（如果有），然后是缺省值参数(如果有)

```
def func(a, n=n+1):  
    print(a, n)
```

```
def f(a, x=0, y=1, z):  
    # 异常SyntaxError: non-default argument follows default argument  
    print(a, x, y, z)
```

函数定义：位置形参和默认值形参

- 函数定义时可包含位置形参和默认值形参
- 函数调用时可通过位置实参和关键字实参传递参数
 - **关键字参数可以按参数名字**传递值，实参顺序可以和形参顺序不一致
 - 在函数定义中有多个默认值参数时通过关键字参数可以明确传递哪些参数，哪些采用默认值
- 默认值形参和关键字实参的语法类似，相互独立的，没有必然的联系
 - 函数定义时可不采用默认值参数，但可通过关键字参数调用
 - 函数定义时可采用默认值参数，但调用时可选择采用关键字参数或者不采用关键字参数传递

```
def func(a, b, c, x='x', y='y'):  
    print(a, b, c, x, y)
```

```
func(1, 2, 3)
```

```
func(1, 2, 3, y=5)
```

函数定义: position-only 参数

python3.8引入position-only参数, 只能通过位置实参传递的参数。经常出现在那些作为API的函数中

- 位置形参和默认值形参都可以通过位置或关键字实参传递参数
- 函数定义时, 某个参数为/, 之前的参数 (可以是默认值参数) 都是position-only参数
 - 不允许通过关键字实参传递。以后改变形参名时不需要修改以前函数调用的代码
 - 以前的规则不变: 默认值参数之后不能再有位置参数

```
>>> help(isinstance)
Help on built-in function isinstance in module builtins:

isinstance(obj, class_or_tuple, /)
...
>>> def f(a, b=3, /, c): pass
SyntaxError: non-default argument follows default argument
>>> def f(a, b, /, c=4, d=5): pass
>>> f(1, 2)
>>> f(1, 2, 5)
>>> f(1, 2, c=5)
>>> f(1, c=5, b=2)
Traceback (most recent call last):
  File "<pyshell#30>", line 1, in <module>
    f(1, c=5, b=2)
TypeError: f() got some positional-only arguments passed as keyword arguments: 'b'
```

函数定义：可变长参数

字典(dict)的每个元素为key:value对
比如 {'name': 'tony', 'gender': 'male', 'age': 20}

可变长度参数主要有两种形式：

*parameter：将那些**尚没有匹配形参的位置实参**组合成一个**元组**赋值给可变长度位置参数

parameter：将那些尚没有匹配形参的关键字实参**组合成一个**字典**赋值给可变长度关键字参数。字典的key为字符串，其内容为关键字实参名，而值为传递的实参对象

- 如果存在**可变长度位置参数**，*name形式的参数应该**出现在默认值参数之后**
- 如果**存在可变长度关键字参数**，**name形式的参数**应该出现在最后**
- 每种类型的可变长度参数只能出现一次
- 函数定义时：位置参数 → 默认值参数 → 可变长度位置参数 → **keyword-only参数** → 可变长度关键字参数

```
def variable_parameter(a, b=4, *x, **y):  
    return (a, b, x, y)
```

```
def test_variable_parameter():  
    print(variable_parameter(1))  
    print(variable_parameter(1,2))  
    print(variable_parameter(1,2,3,4,5))  
    print(variable_parameter(1,u=11,v=12,w=13))  
    print(variable_parameter(1,2,3,4,5,u=11,v=12,w=13))
```

variable_parameter.py

```
(1, 4, (), {})  
(1, 2, (), {})  
(1, 2, (3, 4, 5), {})  
(1, 4, (), {'u': 11, 'v': 12, 'w': 13})  
(1, 2, (3, 4, 5), {'u': 11, 'v': 12, 'w': 13})
```


函数定义：形参与实参之间的匹配

函数定义中的形参顺序：如果某个参数为/，则之前的位置参数（包括默认值参数）仅允许位置实参传递。

位置参数 → 默认值参数 → 可变长度位置参数 → 仅允许关键字传递参数(可定义默认值，也可不定义) → 可变长度关键字参数

- 函数调用中的实参顺序：位置参数 → 关键字参数
- 函数调用时，根据传递的实参按照以下规则匹配形参：
 - 首先按照**位置**匹配形参中的位置参数和默认值参数
 - 接下来根据**关键字参数的名字**匹配形参中的位置参数和默认值参数
 - 函数定义中**尚未匹配的默认值参数**设置为**默认值**
 - 剩下的**位置参数组成元组(tuple)**赋值给**可变长度位置参数**（如果有）
 - 剩下的**关键字参数组成字典(dict)**赋值给**可变长度关键字参数**（如果有）
 - 如果仍然有**尚未匹配**的形参和实参则报错
 - 每个参数**只能匹配一次**

```
def func(pos1, pos2=2, /, pk1="pk1", pk2="pk2",  
        *args, kw1="kw1", kw2, **kwargs):  
    print(pos1, pos2, pk1, pk2, args, kw1, kw2, kwargs, sep=", ")  
  
func(1, 2, 3, 4, 5, 6, 7, kw2=2, k1=1, k3=4, k5=7)
```

函数调用时，根据传递的实参按照以下规则匹配形参：

- 首先按照**位置**匹配形参中的位置参数和默认值参数
- 接下来根据**关键字参数的名字**匹配形参中的位置参数和默认值参数
- 函数定义中**尚未匹配的默认值参数**设置为**默认值**
- 剩下的**位置参数组成元组(tuple)**赋值给**可变长度位置参数（如果有）**
- 剩下的**关键字参数组成字典(dict)**赋值给**可变长度关键字参数（如果有）**
- 如果仍然有**尚未匹配**的形参和实参则报错
- 每个参数**只能匹配一次**

```
def func2(a, b, c, x='x', y='y'):
    print(a, b, c, x, y)
```

在交互式控制台执行

```
>>> func2(1, 2)
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
TypeError: func2() missing 1 required positional argument: 'c'
```

```
>>> func2(1, 2, 3)
```

```
1 2 3 x y
```

```
>>> func2(1, 2, 3, 4)
```

```
1 2 3 4 y
```

```
>>> func2(1, 2, 3, 4, 5)
```

```
1 2 3 4 5
```

```
>>> func2(1, 2, 3, 4, 5, 6)
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
TypeError: func2() takes from 3 to 5 positional arguments but 6 were given
```

```
>>> func2(1, 2, 3, y=5)
```

```
1 2 3 x 5
```

```
>>> func2(b=2, a=1, c=3, y=5)
```

```
1 2 3 x 5
```

```
>>> func2(1, b=2, c=3, x=5)
```

```
1 2 3 5 y
```

```
>>> func2(1, 2, b=2, x=5)
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
TypeError: func2() got multiple values for argument 'b'
```

函数定义：形参与实参之间的匹配

variable_parameter.py

函数调用时，根据传递的实参按照以下规则匹配形参：

- 首先按照**位置**匹配形参中的位置参数和默认值参数
- 接下来根据**关键字参数的名字**匹配形参中的位置参数和默认值参数
- 函数定义中**尚未匹配的默认值参数**设置为**默认值**
- 剩下的**位置参数组成元组(tuple)**赋值给**可变长度位置参数**（如果有）
- 剩下的**关键字参数组成字典(dict)**赋值给**可变长度关键字参数**（如果有）
- 如果仍然有**尚未匹配**的形参和实参则报错
- 每个参数**只能匹配一次**

```
def variable_parameter(a, b=4, *x, **y):  
    return (a, b, x, y)  
  
def test_variable_parameter():  
    print(variable_parameter(1))  
    print(variable_parameter(1,2))  
    print(variable_parameter(1,2,3,4,5))  
    print(variable_parameter(1,u=11,v=12,w=13))  
    print(variable_parameter(1,2,3,4,5,u=11,v=12,w=13))
```

```
(1, 4, (), {})  
(1, 2, (), {})  
(1, 2, (3, 4, 5), {})  
(1, 4, (), {'u': 11, 'v': 12, 'w': 13})  
(1, 2, (3, 4, 5), {'u': 11, 'v': 12, 'w': 13})
```

函数定义：仅允许关键字传递的形参 (keyword-only parameters)

Python3还引入了**keyword-only形参**，**在调用时只能通过关键字实参**的形式来传递

- keyword-only参数**出现在可变长度的位置参数之后，可变长度关键字参数(如果有) 之前**
 - 如果没有可变长度位置参数，则添加一个参数*
- keyword-only参数也可以指定默认值，即支持name或name=default方式
- 调用时必须**采取关键字参数传递**，或者**不传递时**表示采用默认值(如果定义时有默认值)

```
def kwonly(a, *b, c, d=5):  
    return (a, b, c, d)  
  
def kwonly2(a, b, *, c, d=5):  
    return (a, b, c, d)  
  
print(kwonly(1, 2, c=3))  
print(kwonly2(1, 2, c=3, d=4))
```

kwonly.py

```
m = max(4, 5, 6)  
n = min(4, 5, 6)  
print(m, n, sep=', ', end='\n')
```

keyword-only



函数参数总结

- /之前的参数为position-only, 只能通过位置实参传递
- *或者*args后的参数为keyword-only, 只能通过关键字实参传递
- *args和**kwargs分别用来收集多余的位置实参 (组成元组) 和关键字实参 (组成字典)
- 默认值参数之后不允许有普通的位置参数
- 关键字实参之后不允许有位置实参

func(position-only, /, position-or-keyword, *args, keyword-only, **kwargs)

仅位置参数传递 位置或关键字参数传递 仅关键字参数传递

```
def func(pos1, pos2=2, /, pk1="pk1", pk2="pk2",
        *args, kw1="kw1", kw2, **kwargs):
    print(pos1, pos2, pk1, pk2, args, kw1, kw2, kwargs, sep=", ")

func(1, 2, 3, 4, 5, 6, 7, kw2=2, k1=1, k3=4, k5=7)
```

1, 2, 3, 4, (5, 6, 7), kw1, 2, {'k1': 1, 'k3': 4, 'k5': 7}

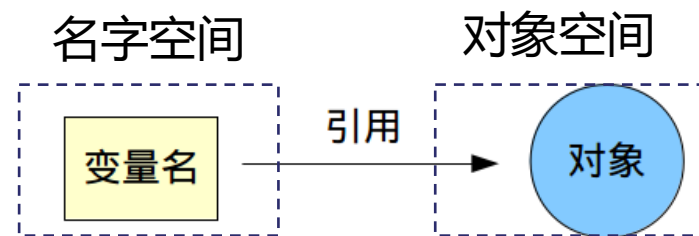
主要内容

- 函数定义：
 - 文档字符串
 - 参数传递
 - return语句
- 函数的参数：
 - 位置实参和关键字实参（格式化时已经介绍）
 - 位置形参和默认值形参
 - Position-only参数
 - 可变长参数
 - Keyword-only参数
- **名字空间：**
 - 内置、全局和本地名字空间
 - 赋值类语句、global和nonlocal语句
 - 未限定名的名字空间搜索：LEGB
- 类型提示

名字空间(namespace)

名字空间是名字（变量）和对象的映射。三个问题：

- 有哪些类型的名字空间？相应名字的作用域？
 - 如果一个名字前面不限定名字空间，通过该名字可以访问的（代码范围）称为名字的作用域
- 通过赋值类语句引入的名字属于哪个名字空间？
 - 赋值语句
 - import语句
 - def语句
 - 参数传递
 - for循环中的for变量
- 表达式中出现未限定名时，该名字是哪个名字空间的名字？



问题1：3种名字空间

locals()	当前名字空间的名字与对象的映射
globals()	全局名字空间的名字与对象的映射
vars([obj])	无参数时等价于locals(), 否则obj的名字空间

- 顶层名字空间: 全局名字空间和内置名字空间
 - **全局名字空间**也称为模块名字空间:
 - 模块被加载(import)或者以脚本方式运行时创建
 - 不属于任何函数的函数体的赋值类语句引入的名字, 称为**全局变量**
 - globals(): 返回字典对象, 其key为名字, 而value为对应的对象
 - 全局变量的作用域为定义它的模块
 - **内置名字空间**: 解释器运行时其内部所加载的builtins模块的名字空间
 - 在进入解释器时创建, 退出解释器时结束: import builtins as __builtins__
 - 内置名字空间的名字就是__builtins__保存的模块对象的属性, 包含了内置函数以及异常类型
 - dir(__builtins__)可以查看该名字空间的名字
 - vars(__builtins__): 查看内置名字空间
- **本地名字空间**: 函数调用时创建的临时的名字空间, 函数调用结束时释放
 - 函数体内赋值类语句引入 (当前函数体没有通过global或nonlocal语句声明时) 的名字, 称为本地或**局部变量**
 - 局部变量作用域为 (本次调用的) 当前函数体
 - locals(): 返回当前名字空间。函数体内调用时返回本地名字空间

问题1：3种名字空间：函数嵌套

- 允许函数嵌套：函数体内也可再定义一个新的函数
- 本地名字空间的名字的作用域是（本次调用的）当前函数体
 - 意味着在内层函数的函数体内也可以访问外层(enclosing)函数的本地名字空间的名字

```
def quadratic_function(a, b, c):  
    """一元二次函数:  $f(x) = ax^2 + bx + c$  """  
    if a == 0:  
        raise ValueError("a cannot be zero")  
  
    def f(x):  
        return a * x**2 + b * x + c  
  
    return f  
  
f = quadratic_function(2, 3, -5)  
print(f(2)) # 输出: 9
```

名字空间

内置：在python环境的内置模块**builtins**中定义的变量（内置函数等）

全局(模块)：在函数外部定义的变量或者在函数体内通过**global**声明过

外层：外层函数内部赋值的变量或通过**nonlocal**声明过，往外层搜索

本地：函数内部赋值的变量，且没有通过**global/nonlocal**声明过

问题2：赋值类语句引入的名字属于哪个名字空间？

- 如果赋值类语句出现的位置不属于任意一个函数的函数体，则定义的名字 var 属于模块（全局）名字空间
- 如果赋值类语句出现在某个函数(如func)的函数体
 - 在func的函数体是否有global或nonlocal语句？
 - 如果有global var，则定义的名字属于全局(模块)名字空间
 - 如果有nonlocal var，则定义的名字属于外层函数的名字空间
 - 从当前函数往外查找外层函数中是否有相应的名字。定义的名字属于第一个查找到的该外层函数的名字空间
 - 否则定义的名字属于当前函数的（本次调用创建的）名字空间

名字空间

内置：在python环境的内置模块**builtins**中定义的变量（内置函数等）

全局(模块)：在函数外部定义的变量或者在函数体内通过**global**声明过

外层：外层函数内部赋值的变量或通过**nonlocal**声明过，往外层搜索

本地：函数内部赋值的变量，且没有通过**global/nonlocal**声明过

legb.c

```
def foo():
    def bar():
        def foobar():
            nonlocal a
            a = "a@foobar"
            nonlocal b
            b = "b@foobar"
            global c
            c = "c@foobar"

        b = "a@bar"
        foobar()

    a = "a@foo"
    bar()

if __name__ == "__main__":
    foo()
```

问题3：名字搜索规则：LEGB

- 迟来绑定策略：只有在对表达式进行求解的时候，才会真正去相应的名字空间查找得到对应的对象
- 根据名字出现的位置，遵循LEGB(Local-Enclosing-Global-Builtin)的原则
 - 直到到达限定的名字空间范围，或找到匹配为止
 - 如果最终没有找到，则抛出异常NameError
- 表达式出现在函数体外：限定顶层名字空间 (global--> builtin) 查找
- 表达式出现在函数体内：
 - 函数体是否有相关global、nonlocal和赋值类语句？
 - global语句：限定顶层名字空间查找
 - 通过赋值类语句更新时，更新的是全局名字空间的名字
 - nonlocal语句：限定仅在外层名字空间查找，不会在顶层名字空间中查找
 - 在分析函数体的时候就已经确定了是外层哪个名字空间（从内层往外层方向的第一个匹配的名字）的名字
 - 通过赋值类语句更新时，更新外层某个名字空间的名字
 - 赋值类语句：其先后顺序并不重要
 - 限定仅在本地名字空间查找
 - 通过赋值类语句更新时，更新的是本地名字空间的名字
 - 否则按照顺序在Enclosing-Global-Builtin 名字空间查找

名字空间

内置：在python环境的内置模块**builtins**中定义的变量（内置函数等）

全局(模块)：在函数外部定义的变量或者在函数体内通过**global**声明过

外层：外层函数内部赋值的变量或通过**nonlocal**声明过，往外层搜索

本地：函数内部赋值的变量，且没有通过**global/nonlocal**声明过

示例：global语句

- 函数log1被调用时，本地名字空间：msg和msg_id
 - msg_id相关的赋值类语句导致仅仅在本地名字空间中查找
 - 尽管赋值类语句出现在表达式访问之后，也是如此
- 函数log2引入global语句
 - 表达式中出现msg_id时：顶层名字空间查找
 - 函数体内通过赋值类语句更新msg_id时，更新的是全局名字空间的名字
- 函数log3: 不建议的用法
 - globals(): 返回全局名字空间的名字和对象的映射
 - sys.modules保存模块名与模块对象之间的映射
 - 全局名字空间的名字就是模块对象的属性

```
logger = 'MAIN'
msg_id = 1
```

```
def log1(msg):
    print(f'[{logger}:{msg_id}] {msg}')
    msg_id += 1
```

```
>>> log1('first msg') # 在交互式控制台执行
```

```
Traceback (most recent call last):
```

```
...
```

```
UnboundLocalError: cannot access local variable 'msg_id'
where it is not associated with a value
```

```
def log2(msg):
    global msg_id
    print(f'[{logger}:{msg_id}] {msg}')
    msg_id += 1
```

```
>>> log2('first msg') # 在交互式控制台执行
```

```
[MAIN:1] first msg
```

```
>>> log2('second msg')
```

```
[MAIN:2] second msg
```

```
def log3(msg):
    import sys
    print(f'[{logger}:{sys.modules[__name__].msg_id}] {msg}')
    globals()['msg_id'] += 1
```

```
>>> log3('third msg') # 在交互式控制台执行
```

```
[MAIN:3] third msg
```

示例：nonlocal语句

- 调用make_log(...)返回函数对象log，赋值给log4
- 内层函数log要通过赋值语句更改外层函数(make_log)名字空间的msg_id，必须采用nonlocal语句声明
 - 在构造函数对象log时，通过分析函数体确定是外层哪个函数名字空间（从内层往外层方向的第一个匹配的名字，不会在顶层的名字空间中查找）
 - 赋值类语句所赋值的名字为该外层函数名字空间中的名字
- 函数log中，logger是外层函数(make_log)名字空间的logger

```
def make_log(logger):  
    msg_id = 1  
  
    def log(msg):  
        nonlocal msg_id  
        print(f'[{logger}]:{msg_id}] {msg}')        msg_id += 1  
    return log  
  
# 在交互式控制台执行  
>>> log4 = make_log('LOGGER')  
>>> log4('first msg')  
[LOGGER:1] first msg  
>>> log4.__closure__  
(<cell at 0x0000016374EF72B0: str object at 0x0000016374F34D70>,  
<cell at 0x0000016374F339A0: int object at 0x000007FFF817FE348>)  
>>> log4.__closure__[0].cell_contents  
'LOGGER'  
>>> log4.__closure__[1].cell_contents  
2
```

- **【不要求】**自由变量：在某个函数的函数体内访问的位于更外层函数的名字空间的变量(名字)
 - 通过nonlocal语句声明的变量
 - 没有相关global、nonlocal和赋值类语句，但属于某个外层名字空间的变量
 - 函数对象的属性__closure__中保存了自由变量与其对应的外层函数名字空间的名字之间的关联
 - 函数log中，logger和msg_id为自由变量

主要内容

- 函数定义：
 - 文档字符串
 - 参数传递
 - return语句
- 函数的参数：
 - 位置实参和关键字实参（格式化时已经介绍）
 - 位置形参和默认值形参
 - Position-only参数
 - 可变长参数
 - Keyword-only参数
- 名字空间：
 - 内置、全局和本地名字空间
 - 赋值类语句、global和nonlocal语句
 - 未限定名的名字空间搜索：LEGB

- **类型提示**

类型提示(type hint) 【拓展内容】

- 定义函数的时候不需要声明参数的类型，可以支持泛型
- Python3.5引入了类型提示，可以给变量、函数的参数和返回值增加注解(annotation)，使得代码更加容易理解
- 静态检查工具(比如pylance)可以根据类型提示检查可能的类型错误，避免潜在的bug
- 变量的类型提示：在变量名之后添加冒号和变量类型
- 函数的类型提示：
 - 参数名之后添加冒号和参数类型
 - 提示函数头部结束的冒号之前添加 ->，后面给出函数的返回值类型

```
name: str = "Hello"

def say_hi(name: str) -> None:
    print(f"Hi {name}")

say_hi(123)  # 输出Hi 123
```

类型提示示例

type_hints.py

```
from typing import Union

# int|float 或者Union[int, float], 表示整数或浮点数类型
# n: float, n可以是float或int类型
def cubic(n: float) -> float:
    return n**3

cubic(4)
cubic(3.14)

# list[int] 表示一个列表, 其中的元素都是整数
def append_left(v: list[int], n: int) -> None:
    v.insert(0, n)

# dict[str, int] 表示字典的key 为字符串类型, 而value 为整数
def reinit_key(d: dict[str, int], key: str) -> None:
    if key in d:
        d[key] = 0
```